

# SOFT2201 Assignment 3

## Code Review

### The use of OOP

#### Encapsulation

```
/**
 * Represents Ghost entity in Pac-Man Game
 */
3 implementations  👤 ed system on behalf of Kylie Guo ✨ <ed> *
public interface Ghost extends DynamicEntity, PlayerPositionObserver {

    /**
     * Sets the speeds of the Ghost for each GhostMode
     * @param speeds speeds of the Ghost for each GhostMode
     */
    1 usage  2 implementations  👤 ed system on behalf of Kylie Guo ✨ <ed>
    void setSpeeds(Map<GhostMode, Double> speeds);

    /**
     * Sets the mode of the Ghost used to calculate target position
     *
     * @param ghostMode mode of the Ghost
     */
    5 usages  2 implementations  👤 ed system on behalf of Kylie Guo ✨ <ed>
    void setGhostMode(GhostMode ghostMode);

    1 usage  2 implementations  new *
    GhostMode getGhostMode();
    2 implementations  new *
    void setImage(Image image);
}
```

In the `Ghost` interface contains the ghost behaviours by using the setter ( `setSpeeds` ,

`setGhostMode`, `setImage`) and `getter` (`getGhostMode`) methods. The classes can only be accessible through those methods, which helps to keep the internal state changes concealed.

## Inheritance

```
/**
 *  Represents a Static Entity in Pac-Man game
 */
8 usages 3 implementations  ed system on behalf of Kylie Guo ✨ <ed>
public interface StaticEntity extends Renderable {
    /**
     * Returns true if Dynamic Entities can pass through entity
     *
     * @return true, if Dynamic Entities can pass through entity
     */
    1 usage 2 implementations  ed system on behalf of Kylie Guo ✨ <ed>
    boolean canPassThrough();
}
```

```
public class StaticEntityImpl implements Renderable, StaticEntity {

    2 usages
    private final Image image;
    6 usages
    private final BoundingBox boundingBox;
    3 usages
    private Layer layer;

    2 usages  ed system on behalf of Kylie Guo ✨ <ed>
    public StaticEntityImpl(BoundingBox boundingBox, Layer layer, Image image) {
        this.image = image;
        this.layer = layer;
        this.boundingBox = boundingBox;
    }

     ed system on behalf of Kylie Guo ✨ <ed>
    @Override
    public double getWidth() { return this.boundingBox.getWidth(); }
```

The `StaticEntity` and `StaticEntityImpl` displays the ability of inheritance. The `StaticEntity` provides an interface for the static game entities, then the `StaticEntityImpl` inherits attributes and methods from it to implement. This allows developers to utilise code from the parent class while avoiding duplication.

## The use of design principles

### Single Responsibility Principle (SRP) & High Cohesion (GRASP)

```
/**
 * Helper class to read JSONObject to retrieve level configuration details
 */
2 usages  ed system on behalf of Kylie Guo ✨ <ed> *
public class LevelConfigurationReader {

    4 usages
    private final JSONObject levelConfiguration;

    1 usage  ed system on behalf of Kylie Guo ✨ <ed>
    public LevelConfigurationReader(JSONObject levelConfiguration) { this.levelConfiguration = levelConfiguration; }

    /**
     * Retrieves the player's speed for the level
     *
     * @return the player's speed for the level
     */
    1 usage  ed system on behalf of Kylie Guo ✨ <ed>
    public double getPlayerSpeed() { return ((Number) levelConfiguration.get("pacmanSpeed")).doubleValue(); }

    /**
     * Retrieves the lengths of the ghost modes in seconds
     *
     * @return the lengths of the ghost modes in seconds
     */
    1 usage  ed system on behalf of Kylie Guo ✨ <ed> *
    public Map<GhostMode, Integer> getGhostModeLengths() {
        Map<GhostMode, Integer> ghostModeLengths = new HashMap<>();
        JSONObject modeLengthsObject = (JSONObject) levelConfiguration.get("modeLengths");
        ghostModeLengths.put(GhostMode.CHASE, ((Number) modeLengthsObject.get("chase")).intValue());
        ghostModeLengths.put(GhostMode.SCATTER, ((Number) modeLengthsObject.get("scatter")).intValue());
        ghostModeLengths.put(GhostMode.FRIGHTENED, ((Number) modeLengthsObject.get("frightened")).intValue());
        return ghostModeLengths;
    }
}
```

The `LevelConfigurationReader` class only responds to reading and parsing level configurations from a json file, which is ahead of the SRP's requirement that a class serve a single and obvious purpose.

This also maintains the high cohesion that groups all related configuration code into a single, focused class.

### Liskov Substitution Principle (LSP)

```

/**
 * Represents a Static Entity in Pac-Man game
 */
8 usages 3 implementations  ed system on behalf of Kylie Guo ✨ <ed>
public interface StaticEntity extends Renderable {
    /**
     * Returns true if Dynamic Entities can pass through entity
     *
     * @return true, if Dynamic Entities can pass through entity
     */
    1 usage 2 implementations  ed system on behalf of Kylie Guo ✨ <ed>
    boolean canPassThrough();
}

```

```

public class StaticEntityImpl implements Renderable, StaticEntity {

    2 usages
    private final Image image;
    6 usages
    private final BoundingBox boundingBox;
    3 usages
    private Layer layer;

    2 usages  ed system on behalf of Kylie Guo ✨ <ed>
    public StaticEntityImpl(BoundingBox boundingBox, Layer layer, Image image) {
        this.image = image;
        this.layer = layer;
        this.boundingBox = boundingBox;
    }

    ed system on behalf of Kylie Guo ✨ <ed>
    @Override
    public double getWidth() { return this.boundingBox.getWidth(); }
}

```

The `StaticEntityImpl` class can replace any instance of `StaticEntity` without interfering with system functions. This helps with the model good inheritance hierarchies and the implementation of `StaticEntity` can be interchangeably.

## Interface Segregation Principle (ISP) & Low Coupling (GRASP)

```

/**
 * Subject that is being observed by PlayerPositionObserver
 */
4 usages 2 implementations  ed system on behalf of Kylie Guo ✨ <ed>
public interface PlayerPositionSubject {

    /**
     * Adds an observer to list of observers for subject
     *
     * @param observer observer for PlayerPositionSubject
     */
    1 usage 1 implementation  ed system on behalf of Kylie Guo ✨ <ed>
    void registerObserver(PlayerPositionObserver observer);

    /**
     * Removes an observer from list of observers for subject
     *
     * @param observer observer for PlayerPositionObserver
     */
    no usages 1 implementation  ed system on behalf of Kylie Guo ✨ <ed>
    void removeObserver(PlayerPositionObserver observer);

    /**
     * Notifies observer of change in player's position
     */
    1 usage 1 implementation  ed system on behalf of Kylie Guo ✨ <ed>
    void notifyObservers();
}

```

```

/**
 * Observer for PlayerPositionObserver
 */
9 usages 4 implementations  ed system on behalf of Kylie Guo ✨ <ed>
public interface PlayerPositionObserver {

    /**
     * Updates observer with the new position of the player
     *
     * @param position the player's position
     */
    2 implementations  ed system on behalf of Kylie Guo ✨ <ed>
    void update(Vector2D position);
}

```

The splitted observer functionality by `PlayerPositionObserver` and `PlayerPositionSubject` interface represents ISP. This can let the clients easily know about the methods they cared and also reduces unnecessary dependencies.

This ensures the low coupling that permits the interacted class without direct dependencies. This makes easily to modify without any more refactoring.

## Dependency Inversion Principle (DIP)

```

/**
 * Responsible for delegating createRenderable requests to the appropriate registered concrete factory.
 */
7 usages 1 implementation  ed system on behalf of Kylie Guo !* <ed>
public interface RenderableFactoryRegistry {

    /**
     * Instantiates a renderable of the given type with the given position
     *
     * @param renderableType type to create
     * @param position starting coordinate position of renderable
     */
    1 implementation  ed system on behalf of Kylie Guo !* <ed>
    Renderable createRenderable(
        char renderableType,
        Vector2D position
    );

    /**
     * Adds factory to registry based on the type of renderable it creates
     * @param renderableType renderable type it creates
     * @param renderableFactory factory that creates that type of renderable
     */
    13 usages 1 implementation  ed system on behalf of Kylie Guo !* <ed>
    void registerFactory(char renderableType, RenderableFactory renderableFactory);
}

```

The `RenderableFactoryRegistry` is solely applicable to the `RenderableFactory` interface and not used on any other implementations. This follows the DIP, which allows the registry to interact with numerous factories through an abstraction, making the code more flexible and maintainable.

## Controller Pattern (GRASP)

```

public class GameEngineImpl implements GameEngine {
    public void startGame() {}
    public void tick() {}
    // etc...
}

```

The `GameEngineImpl` over this application acts as a controller for managing the game states. This centralises game control by organising important functionalities into a single class.

## Creator Pattern (GRASP)

```

/**
 * Concrete renderable factory for Pac-Man objects
 */
1 usage  ed system on behalf of Kylie Guo ✨ <ed>
public class PacmanFactory implements RenderableFactory {
    2 usages
    private static final Image playerLeftImage = new Image( url: "maze/pacman/playerLeft.png");
    1 usage
    private static final Image playerRightImage = new Image( url: "maze/pacman/playerRight.png");
    1 usage
    private static final Image playerUpImage = new Image( url: "maze/pacman/playerUp.png");
    1 usage
    private static final Image playerDownImage = new Image( url: "maze/pacman/playerDown.png");
    1 usage
    private static final Image playerClosedImage = new Image( url: "maze/pacman/playerClosed.png");

    ed system on behalf of Kylie Guo ✨ <ed>
    @Override
    public Renderable createRenderable(
        Vector2D position
    ) {
        try {
            Map<PacmanVisual, Image> images = new HashMap<>();
            images.put(PacmanVisual.UP, playerUpImage);
            images.put(PacmanVisual.DOWN, playerDownImage);
            images.put(PacmanVisual.LEFT, playerLeftImage);
            images.put(PacmanVisual.RIGHT, playerRightImage);
            images.put(PacmanVisual.CLOSED, playerClosedImage);

            Image currentImage = playerLeftImage;
            position = position.add(new Vector2D( x: 4, y: -4));

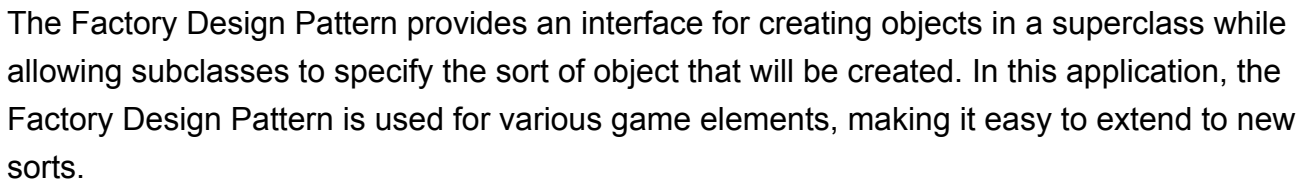
            BoundingBox boundingBox = new BoundingBoxImpl(
                position,

```

The `PacmanFactory` class is role of creating an instance for `Pacman`. This follows the creator pattern by the responsibility for instantiation is delegated to a class that has the necessary information to build `Pacman` objects.

## The use of design patterns

### Factory Design Pattern



### 1. **RenderableFactory** (Interface):

## 2. Concrete Factories:

**BlinkyFactory, ClydeFactory, InkyFactory, PinkyFactory**: Each creates a different type of ghost by applying the `ChaseStrategy` behaviours.

Manage a registry of factories that allows to retrieve the appropriate factory for each `RenderableType`.

Defines constants for different types of renderable entities via char variables.

```
classDiagram
    class KeyboardInputHandler
    class MovementInvokerSingleton {
        - instance: MovementInvoker
        + getInstance(): MovementInvoker
    }
    KeyboardInputHandler --> MovementInvokerSingleton
```

The Singleton Design Pattern ensures that a class has only one instance and provides a global access. Over here is one of the Singleton Design Pattern used in this application. The

`MovementInvoker` class uses the Singleton to control movement and ensures only one instance of the `MovementInvoker` is created.

## Participants and Relationships

### 1. `MovementInvokerSingleton` (Singleton Class):

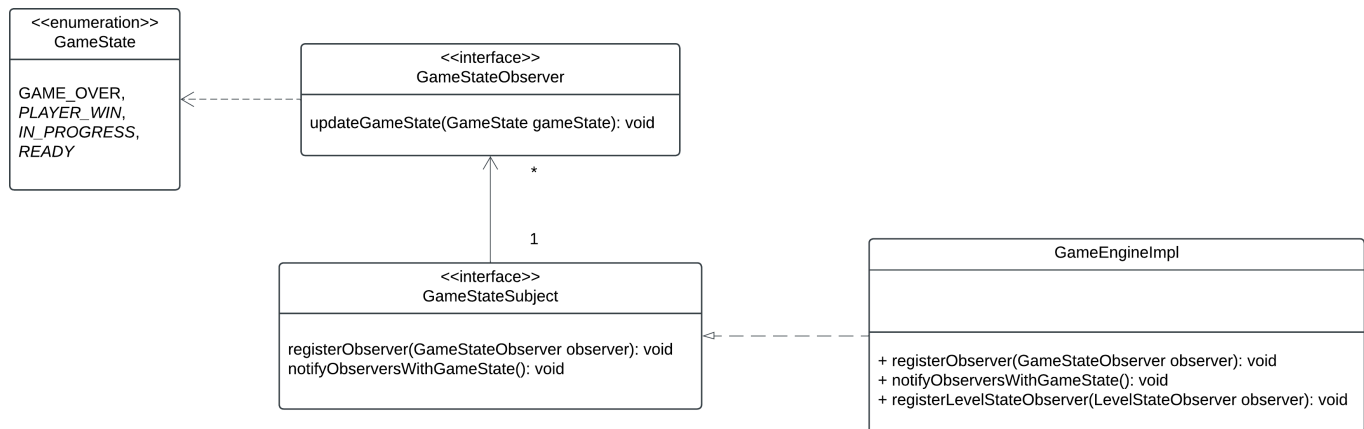
Holds a private static instance of `MovementInvoker`.

Provides a static `getInstance()` method that is used to return the single instance of `MovementInvoker`.

### 2. `KeyboardInputHandler` (Client):

Interacts with the `MovementInvokerSingleton` to use the `MovementInvoker` instance to coordinate movement.

## Observer Design Pattern



The Observer Design Pattern sets up an one-to-many relationship between objects, which the changes will automatically notify and update dependent objects. In the game, the pattern is applied on game states and then notify observers of tracking.

## Participants and Relationships

### 1. `GameStateSubject` (Interface):

Defines methods for registering observers ( `registerObserver(GameStateObserver observer)` ) and notifying them ( `notifyObserversWithGameState()` ).

### 2. `GameStateObserver` (Interface):

Declares the `updateGameState(GameState gameState)` method, and all observers need to implement to handle updates.

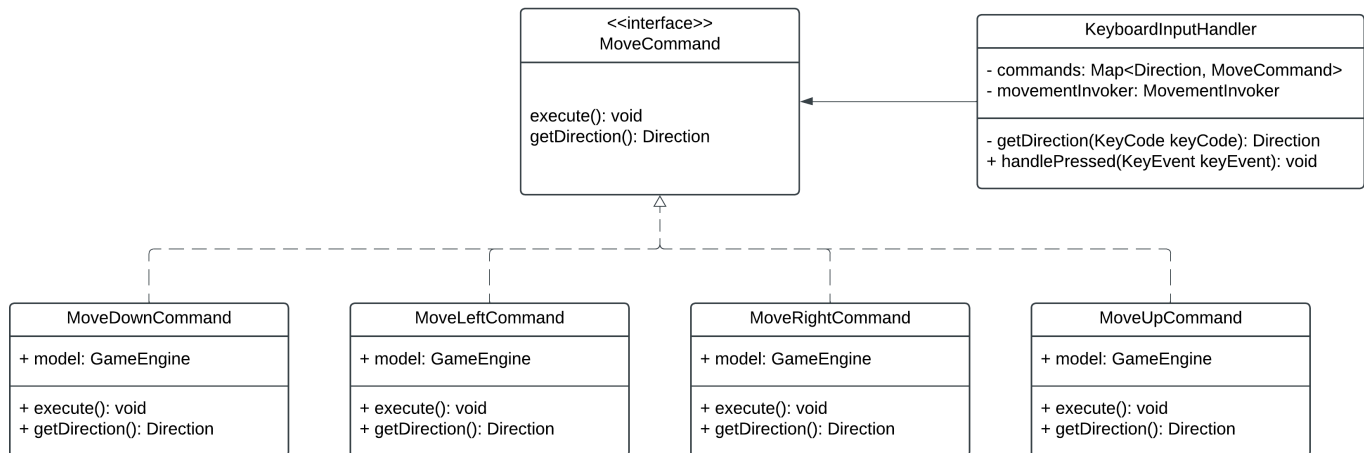
### 3. `GameEngineImpl` (Concrete Subject):

Implements the `GameStateSubject` interface and manages `GameStateObserver` instances.

### 4. `GameState` (Enumeration):

Handle the various game states.

# Command Design Pattern



The Command Pattern allows parameterisation of clients with different requests. In this game, the Command Design Pattern is defined to handle the keyboard inputs for movement, and each command represents a direction.

## Participants and Relationships

### 1. MoveCommand (Interface):

execute() method for performing the movement action and getDirection() for retrieving the corresponding direction.

### 2. Concrete Commands:

**MoveDownCommand** : handle the logic of move down

**MoveLeftCommand** : handle the logic of move left

**MoveRightCommand** : handle the logic of move right

**MoveUpCommand** : handle the logic of move up

### 3. KeyboardInputHandler (Invoker):

Associates directions with the respective MoveCommand .

## Documentation

### Comments

The comment implemented in an application is often used to determine a human-readable explanation to allow other developers to understand the purpose of making the code. Here is the comments existed in this game:

### In-line comments

- Explain the logic of the if-statement that is used to continue in a direction for ghosts before changing direction.

```
// ghosts have to continue in a direction for a minimum time before changing direction
if (currentDirection != null && currentDirectionCount < minimumDirectionCount) {
    currentDirectionCount++;
    return currentDirection;
}
```

- Explain the logic to set up the map and get the level configuration.

```
1 usage  ed system on behalf of Kylie Guo ✨ <ed>
private void init(GameConfigurationReader gameConfigurationReader) {
    // Set up map
    String mapFile = gameConfigurationReader.getMapFile();
    MazeCreator mazeCreator = new MazeCreator(mapFile, renderableFactoryRegistry);
    this.maze = mazeCreator.createMaze();
    this.maze.setNumLives(gameConfigurationReader.getNumLives());

    // Get level configurations
    this.levelConfigs = gameConfigurationReader.getLevelConfigs();
    this.numLevels = levelConfigs.size();
    if (levelConfigs.isEmpty()) {
        System.exit(status: 0);
    }
}
```

- Explain the logic of resetting renderables to the starting state.

```
2 usages  ed system on behalf of Kylie Guo ✨ <ed>
private void startLevel() {
    JSONObject levelConfig = (JSONObject) levelConfigs.get(currentLevelNo);
    // reset renderables to starting state
    maze.reset();
    this.currentLevel = new LevelImpl(levelConfig, maze);
    for (LevelStateObserver observer : this.levelStateObservers) {
        this.currentLevel.registerObserver(observer);
    }
    this.setGameState(GameState.READY);
}
```

- Explain the logic that to remove the observers.

```
1 usage  ed system on behalf of Kylie Guo ✨ <ed>
private void handleLevelEnd() {
    if (numLevels - 1 == currentLevelNo) {
        handlePlayerWins();
    } else {
        this.currentLevelNo += 1;

        // remove observers
        for (LevelStateObserver observer : this.levelStateObservers) {
            this.currentLevel.removeObserver(observer);
        }

        startLevel();
    }
}
```

- Explain the logic for updating ghost directions.

```
1 usage  ed system on behalf of Kylie Guo ✨ <ed>
private void updateDirection() {
    // Ghosts update their target location when they reach an intersection
    if (Maze.isAtIntersection(this.possibleDirections)) {
        this.targetLocation = getTargetLocation();
    }

    Direction newDirection = selectDirection(possibleDirections);

    // Ghosts have to continue in a direction for a minimum time before changing direction
    if (this.currentDirection != newDirection) {
        this.currentDirectionCount = 0;
    }
    this.currentDirection = newDirection;

    switch (currentDirection) {
        case LEFT -> this.kinematicState.left();
        case RIGHT -> this.kinematicState.right();
        case UP -> this.kinematicState.up();
        case DOWN -> this.kinematicState.down();
    }
}
```

- Explain the logic of ghosts selecting direction.

```
1 usage 2 ed system on behalf of Kylie Guo 3 <ed>
private Direction selectDirection(Set<Direction> possibleDirections) {
    if (possibleDirections.isEmpty()) {
        return currentDirection;
    }

    // ghosts have to continue in a direction for a minimum time before changing direction
    if (currentDirection != null && currentDirectionCount < minimumDirectionCount) {
        currentDirectionCount++;
        return currentDirection;
    }

    Map<Direction, Double> distances = new HashMap<>();

    for (Direction direction : possibleDirections) {
        // ghosts never choose to reverse travel
        if (currentDirection == null || direction != currentDirection.opposite()) {
            distances.put(direction, Vector2D.calculateEuclideanDistance(this.kinematicState.getPotentialPosition(direction), this.kinematicState.getPosition()));
        }
    }

    // only go the opposite way if trapped
    if (distances.isEmpty()) {
        return currentDirection.opposite();
    }

    // select the direction that will reach the target location fastest
    return Collections.min(distances.entrySet(), Map.Entry.comparingByValue()).getKey();
}
```

- Explain the logic of ghost backing to the starting position.

```
@Override
public void reset() {
    // return ghost to starting position
    this.kinematicState = new KinematicStateImpl.KinematicStateBuilder()
        .setPosition(startingPosition)
        .build();
    this.boundingBox.setTopLeft(startingPosition);
    this.ghostMode = SCATTER;
    this.currentDirectionCount = minimumDirectionCount;
}
```

## Javadocs

The Javadoc is a documentation tool for Java language that generates HTML documentation. This can easily parse the declaration for the range of files and methods to lead other developers read the source quickly.

As the below shown, when the Javadoc with the method and attributes description added into the application, the runs the command:

```
`javadoc -d docs -sourcepath src -subpackages pacman`
```

This can quickly generates a HTML page including all folder with the written Javadocs.

```
/**
 * Represents something that can be rendered in Pac-Man
 */
12 implementations  ed system on behalf of Kylie Guo ✨ <ed>
public interface Renderable {

    /**
     * @return image of renderable
     */
    4 implementations  ed system on behalf of Kylie Guo ✨ <ed>
    Image getImage();

    /**
     * @return width of renderable image
     */
    4 implementations  ed system on behalf of Kylie Guo ✨ <ed>
    double getWidth();

    /**
     * @return width of renderable height
     */
    4 implementations  ed system on behalf of Kylie Guo ✨ <ed>
    double getHeight();

    /**
     * Returns the current (x, y) position of this Entity
     *
     * @return The x position using the top left hand corner as 0,0 and increasing as the position moves right on the screen.
     * The y position using the top left hand corner as 0.0 and increasing as the position moves down the screen
     */
}
```

OVERVIEW

PACKAGE

CLASS

TREE

INDEX

HELP

SEARCH:

Packages

| Package                                      | Description |
|----------------------------------------------|-------------|
| pacman                                       |             |
| pacman.model.decorator                       |             |
| pacman.model.engine                          |             |
| pacman.model.engine.observer                 |             |
| pacman.model.entity                          |             |
| pacman.model.entity.dynamic                  |             |
| pacman.model.entity.dynamic.ghost            |             |
| pacman.model.entity.dynamic.physics          |             |
| pacman.model.entity.dynamic.player           |             |
| pacman.model.entity.dynamic.player.observer  |             |
| pacman.model.entity.staticentity             |             |
| pacman.model.entity.staticentity.collectable |             |
| pacman.model.factories                       |             |
| pacman.model.level                           |             |
| pacman.model.level.observer                  |             |
| pacman.model.maze                            |             |
| pacman.model.states                          |             |
| pacman.model.strategies                      |             |
| pacman.view                                  |             |
| pacman.view.background                       |             |
| pacman.view.display                          |             |
| pacman.view.entity                           |             |
| pacman.view.keyboard                         |             |
| pacman.view.keyboard.command                 |             |

And the example of `Renderable` illustrates a method summary with the clearly description and type to allow developer easily find information.

public interface **Renderable**

Represents something that can be rendered in Pac-Man

**Nested Class Summary**

| Nested Classes    |                         |                             |
|-------------------|-------------------------|-----------------------------|
| Modifier and Type | Interface               | Description                 |
| static enum       | <b>Renderable.Layer</b> | The set of available layers |

**Method Summary**

| All Methods              | Instance Methods        | Abstract Methods                                      |
|--------------------------|-------------------------|-------------------------------------------------------|
| Modifier and Type        | Method                  | Description                                           |
| BoundingBox              | <b>getBoundingBox()</b> | Returns the bounding box of the renderable            |
| double                   | <b>getHeight()</b>      |                                                       |
| javafx.scene.image.Image | <b>getImage()</b>       |                                                       |
| Renderable.Layer         | <b>getLayer()</b>       | Returns the current 'z' position to draw this entity. |
| Vector2D                 | <b>getPosition()</b>    | Returns the current (x, y) position of this Entity    |
| double                   | <b>getWidth()</b>       |                                                       |
| void                     | <b>reset()</b>          | Resets the renderable to initial state                |

Here're packages with classes and interfaces for the codebase:

- package pacman.model.engine

**Package pacman.model.engine**

package pacman.model.engine

| Related Packages             |             |
|------------------------------|-------------|
| Package                      | Description |
| pacman.model.engine.observer |             |

| All Classes and Interfaces | Interfaces                                                                    | Classes |
|----------------------------|-------------------------------------------------------------------------------|---------|
| Class                      | Description                                                                   |         |
| GameConfigurationReader    | Helper class to read Game Configuration from JSONObject                       |         |
| GameEngine                 | The base interface for interacting with the Pac-Man model                     |         |
| GameEngineImpl             | Implementation of GameEngine - responsible for coordinating the Pac-Man model |         |

- package pacman.model.engine.observer

## Package pacman.model.engine.observer

package pacman.model.engine.observer

### Related Packages

| Package             | Description |
|---------------------|-------------|
| pacman.model.engine |             |

### All Classes and Interfaces

### Interfaces

### Enum Classes

| Class             | Description                                         |
|-------------------|-----------------------------------------------------|
| GameState         | Represents the possible states of the Pac-Man Game  |
| GameStateObserver | Observer for GameStateSubject                       |
| GameStateSubject  | Subject that is being observed by GameStateObserver |

- package pacman.model.entity

## Package pacman.model.entity

package pacman.model.entity

### Related Packages

| Package                          | Description |
|----------------------------------|-------------|
| pacman.model.entity.dynamic      |             |
| pacman.model.entity.staticentity |             |

### All Classes and Interfaces

### Interfaces

### Enum Classes

| Class            | Description                                          |
|------------------|------------------------------------------------------|
| Renderable       | Represents something that can be rendered in Pac-Man |
| Renderable.Layer | The set of available layers                          |

- package pacman.model.entity.dynamic

## Package pacman.model.entity.dynamic

package pacman.model.entity.dynamic

| Related Packages                    |             |
|-------------------------------------|-------------|
| Package                             | Description |
| pacman.model.entity                 |             |
| pacman.model.entity.dynamic.ghost   |             |
| pacman.model.entity.dynamic.physics |             |
| pacman.model.entity.dynamic.player  |             |
| pacman.model.entity.staticentity    |             |

| Interfaces    |                                                 |
|---------------|-------------------------------------------------|
| Class         | Description                                     |
| DynamicEntity | Represents a dynamic entity in the Pac-Man Game |

- package pacman.model.entity.dynamic.ghost

## Package pacman.model.entity.dynamic.ghost

package pacman.model.entity.dynamic.ghost

| Related Packages                    |             |
|-------------------------------------|-------------|
| Package                             | Description |
| pacman.model.entity.dynamic         |             |
| pacman.model.entity.dynamic.physics |             |
| pacman.model.entity.dynamic.player  |             |

| All Classes and Interfaces | Interfaces                                                                                          | Classes | Enum Classes |
|----------------------------|-----------------------------------------------------------------------------------------------------|---------|--------------|
| Class                      | Description                                                                                         |         |              |
| Ghost                      | Represents Ghost entity in Pac-Man Game                                                             |         |              |
| GhostImpl                  | Concrete implementation of Ghost entity in Pac-Man Game                                             |         |              |
| GhostMode                  | Represents the different modes of ghosts, which determines how ghosts choose their target locations |         |              |

- package pacman.model.entity.dynamic.physics

## Package pacman.model.entity.dynamic.physics

package pacman.model.entity.dynamic.physics

### Related Packages

| Package                            | Description |
|------------------------------------|-------------|
| pacman.model.entity.dynamic        |             |
| pacman.model.entity.dynamic.ghost  |             |
| pacman.model.entity.dynamic.player |             |

### All Classes and Interfaces

#### Interfaces

#### Classes

#### Enum Classes

| Class                                    | Description                                                                                                 |
|------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| BoundingBox                              | Simple bounding volume, represented by a box with no rotation.                                              |
| BoundingBoxImpl                          | Standard implementation of an BoundingBox, with $O(1)$ time collidesWith and containsPoint implementations. |
| Direction                                | Represents the cardinal directions allowed for movement in Pac-Man                                          |
| KinematicState                           | Encapsulation of a dynamic entity's motion.                                                                 |
| KinematicStateImpl                       | Discrete kinematic state implementation.                                                                    |
| KinematicStateImpl.KinematicStateBuilder |                                                                                                             |
| PhysicsEngine                            | Primitive PhysicsEngine implementation.                                                                     |
| Vector2D                                 | Utility object for 2D coordinates.                                                                          |

- package pacman.model.entity.dynamic.player

## Package pacman.model.entity.dynamic.player

package pacman.model.entity.dynamic.player

### Related Packages

| Package                                     | Description |
|---------------------------------------------|-------------|
| pacman.model.entity.dynamic                 |             |
| pacman.model.entity.dynamic.player.observer |             |
| pacman.model.entity.dynamic.ghost           |             |
| pacman.model.entity.dynamic.physics         |             |

### All Classes and Interfaces

#### Interfaces

#### Classes

#### Enum Classes

| Class           | Description                                             |
|-----------------|---------------------------------------------------------|
| Controllable    | Represents a Controllable character in the Pac-Man game |
| MovementInvoker | Handles the execution of moves requested by player      |
| Pacman          |                                                         |
| PacmanVisual    | The possible sprites of Pac-Man                         |

- `package pacman.model.entity.dynamic.player.observer`

## Package `pacman.model.entity.dynamic.player.observer`

`package pacman.model.entity.dynamic.player.observer`

### Related Packages

| Package                                         | Description |
|-------------------------------------------------|-------------|
| <code>pacman.model.entity.dynamic.player</code> |             |

### Interfaces

| Class                               | Description                                                           |
|-------------------------------------|-----------------------------------------------------------------------|
| <code>PlayerPositionObserver</code> | Observer for <code>PlayerPositionObserver</code>                      |
| <code>PlayerPositionSubject</code>  | Subject that is being observed by <code>PlayerPositionObserver</code> |

- `package pacman.model.entity.staticentity`

## Package `pacman.model.entity.staticentity`

`package pacman.model.entity.staticentity`

### Related Packages

| Package                                                   | Description |
|-----------------------------------------------------------|-------------|
| <code>pacman.model.entity</code>                          |             |
| <code>pacman.model.entity.staticentity.collectable</code> |             |
| <code>pacman.model.entity.dynamic</code>                  |             |

### All Classes and Interfaces

#### Interfaces

#### Classes

| Class                         | Description                                |
|-------------------------------|--------------------------------------------|
| <code>StaticEntity</code>     | Represents a Static Entity in Pac-Man game |
| <code>StaticEntityImpl</code> |                                            |

- package pacman.model.entity.staticentity.collectable

## Package pacman.model.entity.staticentity.collectable

package pacman.model.entity.staticentity.collectable

### Related Packages

| Package                          | Description |
|----------------------------------|-------------|
| pacman.model.entity.staticentity |             |

### All Classes and Interfaces

### Interfaces

### Classes

| Class       | Description                                          |
|-------------|------------------------------------------------------|
| Collectable | Represents a collectable entity in the Pac-Man Game. |
| Pellet      | Represents the Pellet in Pac-Man game                |
| PowerPellet |                                                      |

- package pacman.model.factories

## Package pacman.model.factories

package pacman.model.factories

### All Classes and Interfaces

### Interfaces

### Classes

| Class                         | Description                                                                                          |
|-------------------------------|------------------------------------------------------------------------------------------------------|
| BlinkyFactory                 |                                                                                                      |
| ClydeFactory                  |                                                                                                      |
| InkyFactory                   |                                                                                                      |
| PacmanFactory                 | Concrete renderable factory for Pac-Man objects                                                      |
| PelletFactory                 | Concrete renderable factory for Pellet objects                                                       |
| PinkyFactory                  |                                                                                                      |
| PowerPelletFactory            |                                                                                                      |
| RenderableFactory             | Generic factory for Renderables                                                                      |
| RenderableFactoryRegistry     | Responsible for delegating createRenderable requests to the appropriate registered concrete factory. |
| RenderableFactoryRegistryImpl | Responsible for delegating createRenderable requests to the appropriate registered concrete factory. |
| RenderableType                | Mapping of characters used in map text files to renderable type                                      |
| WallFactory                   | Concrete renderable factory for Wall objects                                                         |

- package pacman.model.level

## Package pacman.model.level

package pacman.model.level

### Related Packages

| Package                                     | Description |
|---------------------------------------------|-------------|
| <a href="#">pacman.model.level.observer</a> |             |

### All Classes and Interfaces

#### Interfaces

#### Classes

| Class                                    | Description                                                             |
|------------------------------------------|-------------------------------------------------------------------------|
| <a href="#">Level</a>                    | The base interface for a Pac-Man level.                                 |
| <a href="#">LevelConfigurationReader</a> | Helper class to read JSONObject to retrieve level configuration details |
| <a href="#">LevelImpl</a>                | Concrete implement of Pac-Man level                                     |

- package pacman.model.level.observer

## Package pacman.model.level.observer

package pacman.model.level.observer

### Related Packages

| Package                            | Description |
|------------------------------------|-------------|
| <a href="#">pacman.model.level</a> |             |

### Interfaces

| Class                              | Description                                                                                                |
|------------------------------------|------------------------------------------------------------------------------------------------------------|
| <a href="#">LevelStateObserver</a> | Observer for LevelStateSubject which is interested in the number of lives of the player and the game state |
| <a href="#">LevelStateSubject</a>  | Subject that is being observed by LevelStateObserver                                                       |

- `package pacman.model.maze`

## Package pacman.model.maze

---

`package pacman.model.maze`

### Classes

| Class              | Description                                                     |
|--------------------|-----------------------------------------------------------------|
| <b>Maze</b>        | Stores and manages the renderables for the Pac-Man game         |
| <b>MazeCreator</b> | Responsible for creating renderables and storing it in the Maze |

- `package pacman.view`

## Package pacman.view

---

`package pacman.view`

### Related Packages

| Package                       | Description |
|-------------------------------|-------------|
| <b>pacman</b>                 |             |
| <b>pacman.view.background</b> |             |
| <b>pacman.view.display</b>    |             |
| <b>pacman.view.entity</b>     |             |
| <b>pacman.view.keyboard</b>   |             |

### Classes

| Class             | Description                                    |
|-------------------|------------------------------------------------|
| <b>GameWindow</b> | Responsible for managing the Pac-Man Game View |

- `package pacman.view.background`

## Package `pacman.view.background`

---

`package pacman.view.background`

### Related Packages

| Package                           | Description |
|-----------------------------------|-------------|
| <code>pacman.view</code>          |             |
| <code>pacman.view.display</code>  |             |
| <code>pacman.view.entity</code>   |             |
| <code>pacman.view.keyboard</code> |             |

### All Classes and Interfaces

#### Interfaces

#### Classes

| Class                                 | Description                                         |
|---------------------------------------|-----------------------------------------------------|
| <code>BackgroundDrawer</code>         | Draws background of Game Screen                     |
| <code>StandardBackgroundDrawer</code> | Standard background drawer - draws black background |

- package pacman.view.display

## Package pacman.view.display

package pacman.view.display

### Related Packages

| Package                | Description |
|------------------------|-------------|
| pacman.view            |             |
| pacman.view.background |             |
| pacman.view.entity     |             |
| pacman.view.keyboard   |             |

### All Classes and Interfaces

#### Interfaces

#### Classes

| Class            | Description                                    |
|------------------|------------------------------------------------|
| Display          | Represents Display Node on Pac-Man Game Screen |
| DisplayManager   | Manages the display nodes for Pac-Man          |
| GameStateDisplay | Displays the game status - win, lose, ready    |
| NumLivesDisplay  | Displays the number of lives of player         |
| ScoreDisplay     | Displays the score of the player               |

- `package pacman.view.entity`

## Package `pacman.view.entity`

---

`package pacman.view.entity`

### Related Packages

| Package                             | Description |
|-------------------------------------|-------------|
| <code>pacman.view</code>            |             |
| <code>pacman.view.background</code> |             |
| <code>pacman.view.display</code>    |             |
| <code>pacman.view.keyboard</code>   |             |

### All Classes and Interfaces

#### Interfaces

#### Classes

| Class                       | Description                                   |
|-----------------------------|-----------------------------------------------|
| <code>EntityView</code>     | Responsible for rendering Pac-Man game entity |
| <code>EntityViewImpl</code> | Concrete implementation of EntityView         |

- package pacman.view.keyboard

## Package pacman.view.keyboard

---

package pacman.view.keyboard

### Related Packages

| Package                      | Description |
|------------------------------|-------------|
| pacman.view                  |             |
| pacman.view.keyboard.command |             |
| pacman.view.background       |             |
| pacman.view.display          |             |
| pacman.view.entity           |             |

### Classes

| Class                | Description                                         |
|----------------------|-----------------------------------------------------|
| KeyboardInputHandler | Responsible for handling keyboard input from player |

- `package pacman.view.keyboard.command`

## Package `pacman.view.keyboard.command`

`package pacman.view.keyboard.command`

### Related Packages

| Package                           | Description |
|-----------------------------------|-------------|
| <code>pacman.view.keyboard</code> |             |

### All Classes and Interfaces

### Interfaces

### Classes

| Class                         | Description                           |
|-------------------------------|---------------------------------------|
| <code>MoveCommand</code>      | Movement command player has requested |
| <code>MoveDownCommand</code>  | Move down command                     |
| <code>MoveLeftCommand</code>  | Move left command                     |
| <code>MoveRightCommand</code> | Move right command                    |
| <code>MoveUpCommand</code>    | Move up command                       |

## Extension Rate

This given codebase with the separate clearly folders for each sections that can be easy to find out the target files need to modify. And with the quite large covered Javadocs can be easier to understand the source code declaration. Like to modify the ghost modes into using the state pattern, these can be assist with the modification on the relevant part.

However, some file code is still too messy like `LevelImpl`, `GhostImpl`, etc. with a large range of methods (e.g., `tick()`) that is hard to read and understand the logic. Therefore, this leads to that in this assignment the collision under frightened mode is not be achieved.

## Feature Extension

### Actual Changes and Extensions

#### Changes

## 1. Modification on increasing the time of mode

This is a modification in `LevelImpl` class that is used to increase the time of mode into ticks per second, which helps with the length of mode changing.

```
@Override
public void tick() {
    if (this.gameState != GameState.IN_PROGRESS) {

        if (tickCount >= START_LEVEL_TIME) {
            setGameState(GameState.IN_PROGRESS);
            tickCount = 0;
        }

    } else {
        // modify to increase the time of mode changing
        if (tickCount == modeLengths.get(currentGhostMode)*34) {

            // update ghost mode
            this.currentGhostMode = GhostMode.getNextGhostMode(currentGhostMode);
            for (Ghost ghost : this.ghosts) {
                ghost.setGhostMode(this.currentGhostMode);
            }

            tickCount = 0;
        }
    }
}
```

## 2. Frightened mode timer

This is a modification in `tick()` method on `LevelImpl` class that is used to count the frightened mode timing when the pacman eating a power pellet. As the time up, the ghost needs to change back the original colour.

```
if (frightenedTimer > 0) {
    frightenedTimer--;
    if (frightenedTimer == 0) {
        endFrightenedMode();
    }
}

tickCount++;
```

### 3. Power pellet Collection

This is a modification in `collect()` method on `LevelImpl` class that to add the power pellet collection that is same as the normal pellet collection when be eaten.

```
@Override
public void collect(Collectable collectable) {
    if (collectable instanceof PowerPellet) {
        int eatenPoint = activateFrightenedMode();
        this.points += eatenPoint;
        notifyObserversWithScoreChange(collectable.getPoints());
        this.collectables.remove(collectable);
        return;
    }
    this.points += collectable.getPoints();
    notifyObserversWithScoreChange(collectable.getPoints());
    this.collectables.remove(collectable);
}
```

### 4. Frightened level configuration

This is a modification in `getGhostModeLengths()` and `getGhostSpeeds()` methods on `LevelConfigurationReader` class that is used to retrieve the ghost speeds and mode lengths under Frightened mode via the `config.json` file.

```
ghostModeLengths.put(GhostMode.SCATTER, ((Number) modeLengthsObject.get("scatter")).intValue());
ghostModeLengths.put(GhostMode.FRIGHTENED, ((Number) modeLengthsObject.get("frightened")).intValue());
return ghostModeLengths;
}

/**
 * Retrieves the speeds of the ghosts for each ghost mode
 *
 * @return the speeds of the ghosts for each ghost mode
 */
1 usage  ed system on behalf of Kylie Guo '✚' <ed> *
public Map<GhostMode, Double> getGhostSpeeds() {
    Map<GhostMode, Double> ghostSpeeds = new HashMap<>();
    JSONObject ghostSpeed = (JSONObject) levelConfiguration.get("ghostSpeed");
    ghostSpeeds.put(GhostMode.CHASE, ((Number) ghostSpeed.get("chase")).doubleValue());
    ghostSpeeds.put(GhostMode.SCATTER, ((Number) ghostSpeed.get("scatter")).doubleValue());
    Number frightenedSpeed = (Number) ghostSpeed.get("frightened");
    if (frightenedSpeed != null) {
        ghostSpeeds.put(GhostMode.FRIGHTENED, frightenedSpeed.doubleValue());
    }
    return ghostSpeeds;
}
```

### 5. Frightened enumeration

This is a modification in `GhostMode` enum that to insert a `FRIGHTENED` that to be

determined by the ghosts.

```
public enum GhostMode {  
    14 usages  
    SCATTER,  
    5 usages  
    CHASE,  
    8 usages  
    FRIGHTENED;  
}
```

#### 6. Frightened mode ghost image changes

This is a modification in `setGhostMode()` method on `GhostImpl` class that is used to change frightened image for ghosts when the pacman has eaten a power pellet. After

frightened mode ending, then it automatically switches back to original images.

```
@Override
public void setGhostMode(GhostMode ghostMode) {
    this.ghostMode = ghostMode;
    this.kinematicState.setSpeed(speeds.get(ghostMode));
    // ensure direction is switched
    this.currentDirectionCount = minimumDirectionCount;
    this.currentState = switch (this.ghostMode) {
        case CHASE -> chaseState;
        case SCATTER -> scatterState;
        case FRIGHTENED -> frightenedState;
    };
    // change image
    if (ghostMode.FRIGHTENED == ghostMode){
        this.image = frightened;
    }else {
        this.image = originalImage;
    }
}
```

## 7. New factories registry

This is a modification in `getRenderableFactoryRegistry()` method on `GameEngineImpl` class that is added the new factories into the `registerFactory` to allow to retrieve the new factories.

```
private RenderableFactoryRegistry getRenderableFactoryRegistry() {
    RenderableFactoryRegistry renderableFactoryRegistry = new RenderableFactoryRegistryImpl();
    renderableFactoryRegistry.registerFactory(RenderableType.HORIZONTAL_WALL, new WallFactory(RenderableType.HORIZONTAL_WALL));
    renderableFactoryRegistry.registerFactory(RenderableType.VERTICAL_WALL, new WallFactory(RenderableType.VERTICAL_WALL));
    renderableFactoryRegistry.registerFactory(RenderableType.UP_LEFT_WALL, new WallFactory(RenderableType.UP_LEFT_WALL));
    renderableFactoryRegistry.registerFactory(RenderableType.UP_RIGHT_WALL, new WallFactory(RenderableType.UP_RIGHT_WALL));
    renderableFactoryRegistry.registerFactory(RenderableType.DOWN_LEFT_WALL, new WallFactory(RenderableType.DOWN_LEFT_WALL));
    renderableFactoryRegistry.registerFactory(RenderableType.DOWN_RIGHT_WALL, new WallFactory(RenderableType.DOWN_RIGHT_WALL));
    renderableFactoryRegistry.registerFactory(RenderableType.PELLET, new PelletFactory());
    renderableFactoryRegistry.registerFactory(RenderableType.PACMAN, new PacmanFactory());
    renderableFactoryRegistry.registerFactory(RenderableType.POKEY, new ClydeFactory());
    renderableFactoryRegistry.registerFactory(RenderableType.SHADOW, new BlinkyFactory());
    renderableFactoryRegistry.registerFactory(RenderableType.SPEEDY, new PinkyFactory());
    renderableFactoryRegistry.registerFactory(RenderableType.BASHFUL, new InkyFactory());
    renderableFactoryRegistry.registerFactory(RenderableType.POWER_PELLET, new PowerPelletFactory());
    return renderableFactoryRegistry;
}
```

## Extensions

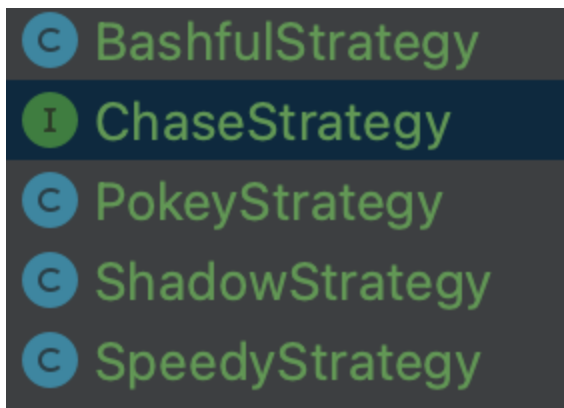
### 1. Additional char variables to deduce the ghost types and power pellets.

This is an extension in `RenderableType` class to determine the ghost types and power pellets through the `map.txt`. This will be used to display different ghosts and power pellets on UI.

```
char PACMAN = 'p';  
2 usages  
char SHADOW = 'b';  
2 usages  
char SPEEDY = 's';  
2 usages  
char BASHFUL = 'i';  
2 usages  
char POKEY = 'c';  
2 usages  
char POWER_PELLET = 'z';
```

### 2. strategies package

This is an extension package that includes the strategy design pattern with the four types of ghost strategies and one interface to be used to implement the `findTargetLoc()` method.



- Ⓒ BashfulStrategy
- Ⓘ ChaseStrategy
- Ⓒ PokeyStrategy
- Ⓒ ShadowStrategy
- Ⓒ SpeedyStrategy

### 3. states package

This is an extension package that includes the state design pattern with the three ghost

modes and one interface to be used to implement the `getTargetLocation()` method.



#### 4. Handling Frightened

This is an extension on `LevelImpl` class that two methods are used to activate and end the Frightened mode when the pacman eating a power pellet.

```
1 usage new *
private int activateFrightenedMode() {
    int eatenPoint = 0;
    for (Ghost ghost : ghosts) {
        ghost.setGhostMode(GhostMode.FRIGHTENED);
        Vector2D respawnPosition = ghost.getPosition();
        ghost = new FrightenedGhostDecorator(ghost, respawnPosition);
        eatenPoint = ((FrightenedGhostDecorator) ghost).getEatenPoints();
    }
    frightenedTimer = modeLengths.get(GhostMode.FRIGHTENED) * 34;
    return eatenPoint;
}

2 usages new *
private void endFrightenedMode() {
    for (Ghost ghost : ghosts) {
        ghost.setGhostMode(GhostMode.SCATTER);
    }
    // to reset
    frightenedTimer = 0;
}
```

#### 5. Power pellet and different ghosts factories

This is an extension on `factories` package that to demonstrate the four type of ghosts and power pellet factories used for other classes and UI display.

```

public class PowerPelletFactory implements RenderableFactory{
    3 usages
    private static final Image PELLET_IMAGE = new Image( url: "maze/pellet.png");
    1 usage
    public static final int DEFAULT_POINTS = 0;
    1 usage
    private final Renderable.Layer layer = Renderable.Layer.BACKGROUND;
    2 usages
    private static final double SIZE_INCREMENT = 2.0;
    1 usage
    private static final Vector2D OFFSET = new Vector2D( x: -8, y: -8);

    new *
    @Override
    public Renderable createRenderable(
        Vector2D position
    ) {
        try {
            // adjust the position with an offset
            Vector2D powerPelletPosition = position.add(OFFSET);
            BoundingBox boundingBox = new BoundingBoxImpl(
                powerPelletPosition,
                height: PELLET_IMAGE.getHeight() * SIZE_INCREMENT,
                width: PELLET_IMAGE.getWidth() * SIZE_INCREMENT
            );
        } catch (Exception e) {
            // handle exception
        }
    }
}

```

```

public class InkyFactory implements RenderableFactory{
    1 usage
    private static final int RIGHT_X_POSITION_OF_MAP = 448;
    1 usage
    private static final int BOTTOM_Y_POSITION_OF_MAP = 16 * 34;

    3 usages
    private static final Image INKY_IMAGE = new Image( url: "maze/ghosts/inky.png");
    1 usage
    Vector2D bottomRightCorner = new Vector2D(RIGHT_X_POSITION_OF_MAP, BOTTOM_Y_POSITION_OF_MAP);
    no usages new *
    private int getRandomNumber(int min, int max) { return (int) ((Math.random() * (max - min)) + min); }
    1 usage
    private ChaseStrategy inkyStrategy = new BashfulStrategy();
    new *
    @Override
    public Renderable createRenderable(
        Vector2D position
    ) {
        try {
            position = position.add(new Vector2D( x: 4, y: -4));

            BoundingBox boundingBox = new BoundingBoxImpl(
                position,
                height: INKY_IMAGE.getHeight() * SIZE_INCREMENT,
                width: INKY_IMAGE.getWidth() * SIZE_INCREMENT
            );
        } catch (Exception e) {
            // handle exception
        }
    }
}

```

## 6. Power pellet class

This is an extension class that represents the power pellet collectable type.

```

package pacman.model.entity.staticentity.collectable;

import ...

new *
public class PowerPellet extends Pellet{
    1 usage new *
    public PowerPellet(BoundingBox boundingBox, Layer layer, Image image, int points) {
        super(boundingBox, layer, image, points);
    }
}

```

## 7. States and strategies instance variables

This is an extension on instance variables that implements the state and strategy patterns on the ghost behaviours.

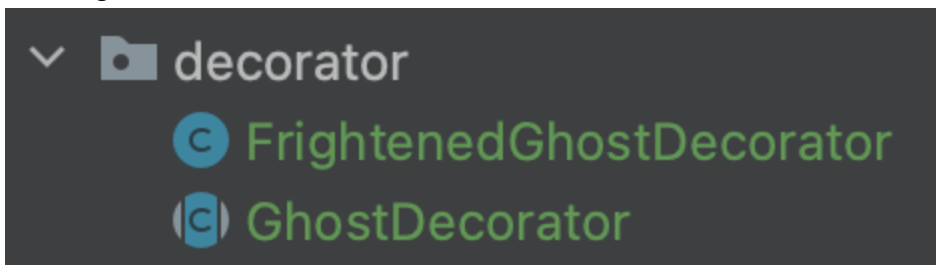
```

public GhostImpl(Image image, BoundingBox boundingBox, KinematicState kinematicState, GhostMode ghostMode, Vect
    this.image = image;
    originalImage = image;
    this.boundingBox = boundingBox;
    this.kinematicState = kinematicState;
    this.startingPosition = kinematicState.getPosition();
    this.ghostMode = ghostMode;
    this.targetCorner = targetCorner;
    this.chaseStrategy = chaseStrategy;
    this.chaseState = new ChaseState(this.chaseStrategy);
    this.scatterState = new ScatterState(this.targetCorner);
    this.frightenedState = new FrightenedState(this.startingPosition);
    this.currentState = switch (this.ghostMode) {
        case CHASE -> chaseState;
        case SCATTER -> scatterState;
        case FRIGHTENED -> frightenedState;
    };
    this.possibleDirections = new HashSet<>();
    this.targetLocation = getTargetLocation();
    this.currentDirection = null;
}

```

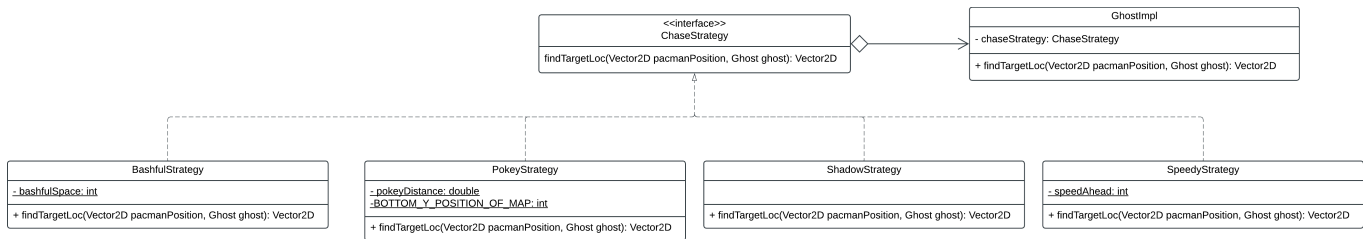
## 8. decorators package

This is an extension package that includes the decorator design pattern for the frightened mode ghost behaviours.



# GoF Design Patterns

## Strategy Design Pattern



The Strategy Design Pattern allows the algorithm to be independently from clients and use it. In this game, the Strategy Pattern is used to manage different ghosts behaviours under CHASE mode with the unique movement.

## Participants and Relationships

### 1. ChaseStrategy (Interface):

Defines the `findTargetLoc(Vector2D pacmanPosition, Ghost ghost): Vector2D` method that each concrete strategy implements to determine the exactly target location on ghosts movement.

### 2. Concrete Strategies:

**BashfulStrategy** : Implements the targeting behaviour for the "Bashful" ghost.

**PokeyStrategy** : Implements the targeting behaviour for the "Pokey" ghost.

**ShadowStrategy** : Implements the targeting behaviour for the "Shadow" ghost.

**SpeedyStrategy** : Implements the targeting behaviour for the "Speedy" ghost.

### 3. GhostImpl (Context):

Contains a reference to a `ChaseStrategy` that permits dynamically to select the strategies.

Call the `findTargetLoc()` to determine each ghost's target action.

## SOLID

### Single Responsibility Principle (SRP)

The Strategy Pattern implements the SRP by separating targets into different classes (`BashfulStrategy`, `PokeyStrategy`, `ShadowStrategy`, and `SpeedyStrategy`). And each class has a single responsibility that leads the different ghost behaviours under the CHASE mode. This prevents the overloaded and focuses on the single management.

### Open/Closed Principle (OCP)

```

4 implementations  new *
public interface ChaseStrategy {
    1 usage  4 implementations  new *
    Vector2D findTargetLoc(Vector2D pacmanPosition, Ghost ghost);
}
  
```

```

2 usages new *
public class PokeyStrategy implements ChaseStrategy{
    1 usage
    private static final double pokeyDistance = 8.0;
    1 usage
    private static final int BOTTOM_Y_POSITION_OF_MAP = 16 * 34;
    1 usage new *
    @Override
    public Vector2D findTargetLoc(Vector2D pacmanPosition, Ghost ghost) {
        double distanceAwayToPacman = Vector2D.calculateEuclideanDistance(ghost.getPosition(), pacmanPosition);
        if (distanceAwayToPacman > pokeyDistance){
            return pacmanPosition;
        } else {
            // go to bottom-left corner
            return new Vector2D(x: 0, BOTTOM_Y_POSITION_OF_MAP);
        }
    }
}

```

The `ChaseStrategy` interface is an open for extensions but closed for modifications. The various behaviours, such as `PokeyStrategy`, can be introduced simply implementing the interface without modifying any current code, allowing for easy growth while keeping existing functionality.

### Liskov Substitution Principle (LSP)

A `ChaseStrategy` interface with the `findTargetLoc()` method can be used interchangeably in `GhostImpl`. This makes sure that `GhostImpl` responses the implementation of `ChaseStrategy` without needing its type.

## GRASP

### Polymorphism

```

4 implementations new *
public interface ChaseStrategy {
    1 usage 4 implementations new *
    Vector2D findTargetLoc(Vector2D pacmanPosition, Ghost ghost);
}

```

The `ChaseStrategy` interface supports polymorphism that allows the program code with different meaning (a range of strategies for ghosts to implement `findTargetLoc`). This enables different ghosts with different behaviours under `CHASE` mode on their specific classes.

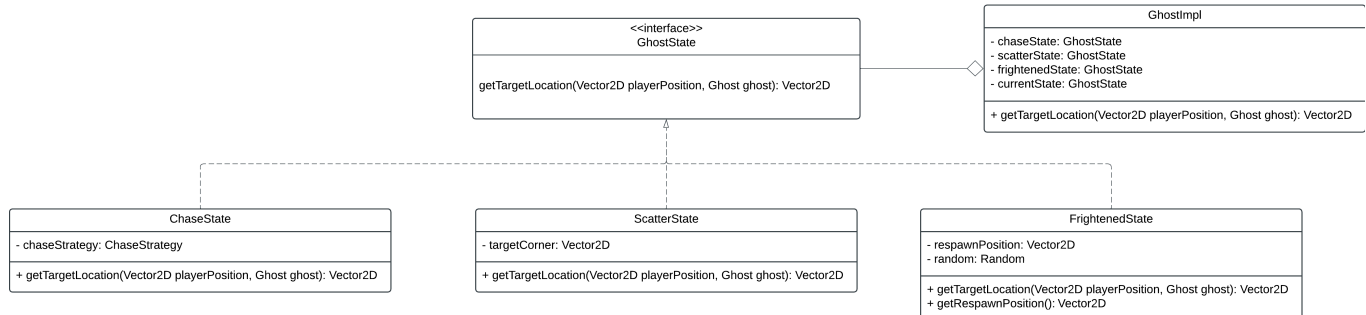
### High Cohesion

The implementation of `ChaseStrategy` contains the high cohesion that is focused on its specific logic. This ensures each strategy only works on a single work and makes debugging easily.

## Low Coupling

The `GhostImpl` only interacts with the `ChaseStrategy` interface, which leads that the changes on `ChaseStrategy` will not affect the `GhostImpl` to keep the low coupling.

## State Design Pattern



The State Pattern allows to change behaviours when its internal state changed. This makes easier to change states independently. In this game, the State Pattern is used to manage different ghost's mode.

## Participants and Relationships

### 1. `GhostState` (Interface):

Declares the `getTargetLocation(Vector2D playerPosition, Ghost ghost): Vector2D` method that provides different targeting logic under the ghost state.

### 2. Concrete States:

**`ChaseState`** : Implements the chasing behaviour and using `chaseStrategy` to calculate Pacman position.

**`ScatterState`** : Implements the scatter behaviour and targets the specific corner.

**`FrightenedState`** : Implements the frightened behaviour and with the additional movement.

### 3. `GhostImpl` (Context):

Holds references to different state instances and tracks the ghost state.

## SOLID

```

package pacman.model.states;

import ...

// define a GhostState change behaviours when its internal state changed
3 implementations new *
public interface GhostState {
    1 usage 3 implementations new *
    Vector2D getTargetLocation(Vector2D playerPosition, Ghost ghost);
}

```

### Single Responsibility Principle (SRP)

The State Pattern implements the SRP by separating targets into different classes (ChaseState, ScatterState and FrightenedState). And each class has a single responsibility that leads the different ghost modes behaviours. This prevents the overloaded and focuses on the single management.

### Open/Closed Principle (OCP)

This supports OCP by allowing the new state class to be added but without modifying existing code. If new more states needed can introduce by implementing the GhostState interface. This ensures the maintainability for the code.

### Liskov Substitution Principle (LSP)

The state classes implement the GhostState interface ensure the substitution without changing the GhostImpl. The GhostImpl can use GhostState interchangeably and ensures predictability in the ghost behaviours.

## GRASP

### Polymorphism

The GhostState interface supports polymorphism that allows the program code with different meaning (a range of states for ghosts mode to implement getTargetLocation). This enables different ghost modes on their specific classes.

```

3 implementations new *
public interface GhostState {
    1 usage 3 implementations new *
    Vector2D getTargetLocation(Vector2D playerPosition, Ghost ghost);
}

```

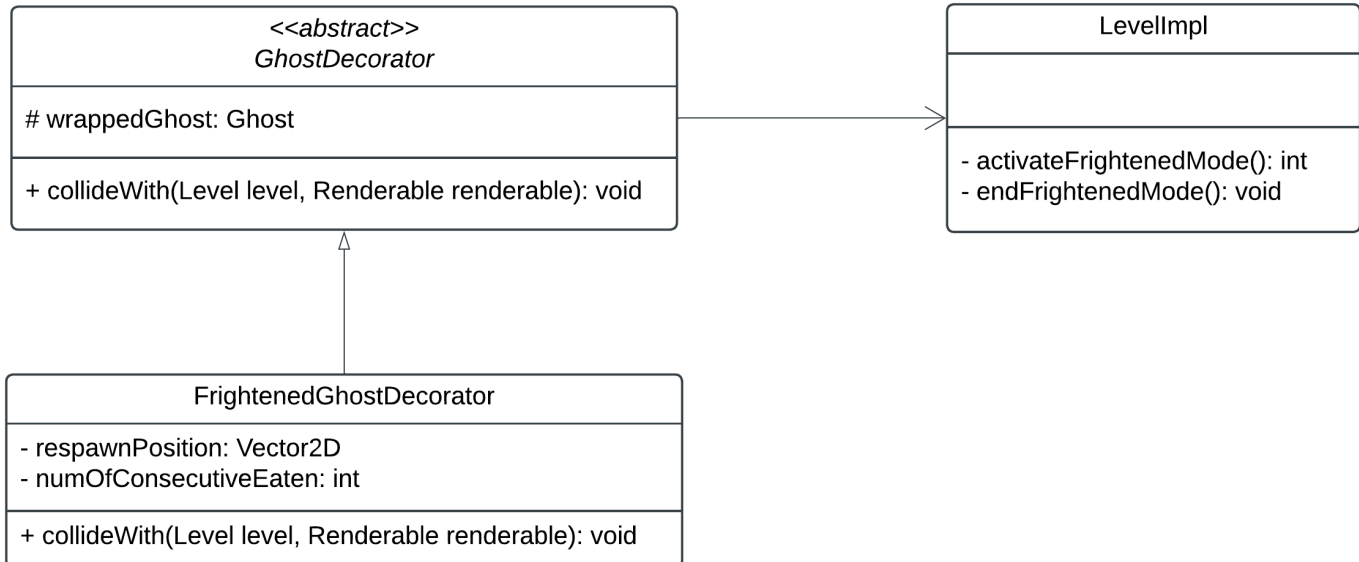
### High Cohesion

The implementation of GhostState contains the high cohesion that is focused on its specific logic. This ensures each strategy only works on a single work and makes debugging easily.

## Low Coupling

The `GhostImpl` only interacts with the `GhostState` interface, which leads that the changes on `GhostState` will not affect the `GhostImpl` to keep the low coupling.

## Decorator Design Pattern



The Decorator Pattern contains an additional responsibilities to an object. This provides flexible on subclassing to extend functionalities. In this game, the decorator pattern is used to modify the ghost behaviours under the `FRIGHTENED` mode and allows the collision.

## Participants and Relationships

### 1. `GhostDecorator` (Abstract Decorator):

Act as a decorator for ghost subjects and holds a reference to a `Ghost`.

### 2. Concrete Decorator:

`FrightenedGhostDecorator`: Extends `GhostDecorator` to add specific behaviours.

### 3. `LevelImpl` (Client):

Contains methods `activateFrightenedMode()` and `endFrightenedMode()` to manage the ghost Frightened mode.

## SOLID

```
// define GhostDecorator method used to add new behaviours to ghost behaviours under frightened mode
1 inheritor new *
public abstract class GhostDecorator implements Ghost {
    4 usages
    protected final Ghost wrappedGhost;

    1 usage new *
    public GhostDecorator(Ghost ghost) { this.wrappedGhost = ghost; }

    4 usages 1 override new *
    @Override
    public void collideWith(Level level, Renderable renderable) { wrappedGhost.collideWith(level, renderable); }
}
}
```

## Single Responsibility Principle (SRP)

The Decorator Pattern implements the SRP by the `GhostDecorator` class and its subclass `FrightenedGhostDecorator`. And each class has a single responsibility: `GhostDecorator` acts as a base decorator for related ghost behaviour; `FrightenedGhostDecorator` handles the ghost behaviours under the `FRIGHTENED` mode. This prevents the overloaded and focuses on the single management.

## Open/Closed Principle (OCP)

This supports OCP that the `FrightenedGhostDecorator` extends `GhostDecorator` without modifying its existing structure. If new more decorators needed can introduce by extending the `GhostDecorator`. This ensures the maintainability for the code.

## Liskov Substitution Principle (LSP)

The inheritance of `FrightenedGhostDecorator` from `GhostDecorator` can be used interchangeably. This enables the frightened behaviour can be used to substitute the normal ghost behaviours under the specific mode.

## Dependency Inversion Principle (DIP)

```
public abstract class GhostDecorator implements Ghost {
    2 usages
    protected final Ghost wrappedGhost;

    1 usage new *
    public GhostDecorator(Ghost ghost) { this.wrappedGhost = ghost; }

    5 usages 1 override new *
    @Override
    public void collideWith(Level level, Renderable renderable) { wrappedGhost.collideWith(level, renderable); }
}
}
```

The `GhostDecorator` is an abstract class that used to implement a decorator design to manage the behaviours of ghosts under `FRIGHTENED` mode. This implies hiding a code's implementation details and only providing the user with the information they need.

## GRASP

## Polymorphism

The decorator pattern supports polymorphism for ghost interactions. Different ghost behaviours can be dynamically applied on the game state and lets the game engine handle interchangeably.

## Protected Variations

The decorator pattern controls the frightened mode and protects the ghost functionality by modifications. This leads the easier modification on frightened ghost behaviours and without affecting on the normal ghost behaviours.

## Low Coupling

The `FrightenedGhostDecorator` requires a reference to the `wrappedGhost` and the level state. This makes the `FrightenedGhostDecorator` easier to modify and debug independently.

## Controller

The `LevelImpl` as a controller manages the ghost decorator behaviours in the frightened mode. The `LevelImpl` coordinates the behaviours to interact with the game entities and called the decorator class.

```
private int activateFrightenedMode() {
    int eatenPoint = 0;
    for (Ghost ghost : ghosts) {
        ghost.setGhostMode(GhostMode.FRIGHTENED);
        Vector2D respawnPosition = ghost.getPosition();
        ghost = new FrightenedGhostDecorator(ghost, respawnPosition);
        eatenPoint = ((FrightenedGhostDecorator) ghost).getEatenPoints();
    }
    frightenedTimer = modeLengths.get(GhostMode.FRIGHTENED) * 34;
    return eatenPoint;
}
```

2 usages   new \*

```
private void endFrightenedMode() {
    for (Ghost ghost : ghosts) {
        ghost.setGhostMode(GhostMode.SCATTER);
    }
    // to reset
    frightenedTimer = 0;
}
```

## Reflection

The extension on the decorator pattern aims to implement the behaviour of ghosts under frightened mode (the collision with pacman then the collided ghost moves back to the starting position) and earns the eaten points. However, the issues over this code is not achieved those behaviour, this only implements the frightened image changes after eating the power pellet.